

MiniPascal presentation and demonstration guide

A practical guide for Swagata: build the executable, enter a small Pascal program, demonstrate output and errors, locate the implementation, and explain the compiler flow in three minutes.

The answers you need first

The correct term is three-address code (TAC). TAC is a form of intermediate representation (IR). In this repository it is generated in **compiler/tac.c**. There is no `ir.c` file.

--run executes the AST through `compiler/interp.c`. **--tac** prints textual IR through `compiler/tac.c`. TAC is not executed by `--run`, and `--tac` does not build a separate executable for the Pascal program.

The `.l` file contains Flex rules. Bison processes `parser.y`, Flex processes `lexer.l`, and GCC compiles the generated C files together with the handwritten C modules into `minipascal.exe`.

Three minute order

0:00 to 0:25 Show `lexer.l`, run the prepared build command, and identify `minipascal.exe`. Rehearse the build beforehand.

0:25 to 0:55 Show `good.pas`, execute it with `--run`, and point to output 4.

0:55 to 1:20 Run `--tac` on the same file. Explain `t1`, `t2`, and `L1`.

1:20 to 1:50 Run the syntax and division-by-zero examples. Show their source locations.

1:50 to 2:35 Show `parser.y` rules, `yyerror`, `interp.c` division check, and `tac.c` `expression/emit_tac`.

2:35 to 3:00 Show the flowchart on page 7 and explain the two branches after the AST.

Prepare the presentation files

Extract the accompanying `MiniPascal_demo_files.zip`. Copy its `presentation_demo` folder into your Desktop repository. All commands below assume PowerShell in that repository. Keep editor tabs open before presenting.

```
cd 'C:\Users\swaga\Desktop\Compiler-Theory-Project - '
```

Source references match the Desktop checkout inspected on 13 September 2026. Line numbers may move after future edits; search terms are supplied as a fallback.

Build the executable and open the source

Open the lexer in the terminal

```
Get-Content .\compiler\lexer.l
notepad .\compiler\lexer.l
```

Get-Content displays it in PowerShell. Notepad opens it for reading. If VS Code is installed, use code . to open the repository and Ctrl+P to choose a file.

Normal project build

```
powershell -ExecutionPolicy Bypass -File .\build.ps1
```

build.ps1 finds Bison, Flex, and GCC or Clang, then runs the generation and compilation steps. The script checks each command exit code. The execution-policy option applies to this launched process.

Manual steps if asked to show the tools

```
win_bison -d -o compiler/parser.tab.c compiler/parser.y
win_flex --wincompat -o compiler/lex.yy.c compiler/lexer.l
gcc -std=gnu11 -Icompiler -o minipascal.exe `
  compiler/parser.tab.c compiler/lex.yy.c `
  compiler/ast.c compiler/dump.c compiler/interp.c `
  compiler/tac.c compiler/main.c
```

These are PowerShell commands: a backtick at the end continues the GCC command. If your tools are named bison and flex, use those names; --wincompat is for win_flex. Do not run lexer.l itself as a program.

If Flex or Bison is missing before presentation

In the environment checked for this guide, GCC was available, but Flex and Bison were not found on PATH. I successfully rebuilt a test executable using the existing generated parser.tab.c and lex.yy.c. For that fallback, run only the GCC command above. It does not regenerate lexer or parser changes.

For a full build, follow the tool-install guidance printed by build.ps1 and reopen the terminal. Confirm all three tools before class. Do not depend on installing tools during the three-minute presentation.

```
Get-Command win_bison, win_flex, gcc
.\minipascal.exe --help
```

If --tac is reported as an unknown option, rebuild: the executable may be older than the source. If any build step fails, stop and fix it before running an old executable as though it were newly built.

Enter a small program and prove it works

Main example in presentation_demo good.pas

```
program Demo;
var n: integer;
begin
  n := 5;
  if n > 0 then n := n - 1;
  writeln(n)
end.
```

To type this yourself, run notepad .\presentation_demo\good.pas, enter the program, and save. Keep the .pas extension. The final end has a period; assignment uses :=.

```
.\minipascal.exe --run .\presentation_demo\good.pas
```

Verified output: 4. The program assigns 5, checks whether it is positive, subtracts 1, and prints the result. The interpreter adds a trailing space after a number; that is normal.

Show tokens and the AST when asked

```
.\minipascal.exe --tokens .\presentation_demo\good.pas
.\minipascal.exe --tree .\presentation_demo\good.pas
```

Tokens include PROGRAM, IDENT, ASSIGN, INTEGER_LIT and IF. The token listing includes line and column. writeln is an IDENT in this lexer; the interpreter recognizes the built-in call name. The tree contains Program, VarDecl, Assign, If, Binary and Call nodes. Although the heading says parse tree, these nodes form the project AST.

If you must type source directly into the terminal

```
.\minipascal.exe --tac
```

Paste the same Pascal source. On a Windows PowerShell console, finish with Ctrl+Z on a new line, then Enter. Terminal behavior can differ, so rehearse this exact sequence. File input is the more predictable fallback; Git Bash EOF can behave differently.

If input means entering a value while running

```
program Demo;
var n: integer;
begin
  readln(n);
  writeln(n + 1)
end.
```

```
.\minipascal.exe --run .\presentation_demo\input.pas
```

Enter 7 and press Enter: verified output is 8. Here the Pascal source comes from a file and readln reads a runtime value. Keep these two kinds of input separate. Enter abc instead to show the numeric-input error on page 5.

Show TAC and locate IR generation

```
.\minipascal.exe --tac .\presentation_demo\good.pas
```

Verified TAC output

```
n = 5
t1 = n > 0
IF_FALSE t1 GOTO L1
t2 = n - 1
n = t2
L1:
WRITE n
Writeln
```

t1 stores the comparison result. If false, control jumps to **L1**. Otherwise **t2** stores $n - 1$ and is copied to n . **WRITE** emits the value and **Writeln** represents a newline. These are printed IR instructions, not the numeric runtime output.

A typical three-address instruction is $\text{result} = \text{operand1 operator operand2}$. In $t2 = n - 1$, the result is $t2$ and the operands are n and 1 . Copy, branch, and I/O instructions can have fewer operands.

Exact code path to show

compiler/main.c lines 145 to 150: `yyparse` checks syntax, `error_count` blocks later work, `--tac` calls `emit_tac(ast_root, stdout)`, and `--run` calls `interpret(ast_root)`.

compiler/tac.c lines 248 to 257: `emit_tac` calls `validate`, lowers statements, and prints buffered instructions with `fputs`. Search `emit_tac`.

compiler/tac.c lines 158 to 164: `N_BINOP` recursively lowers left and right expressions, calls `new_temp`, and emits $\text{result} = \text{left op right}$. This is the clearest answer to where TAC is generated.

```
char *left = expression(t, n->child[0]);
char *right = expression(t, n->child[1]);
char *result = new_temp(t);
/* after checking these values */
emit(t, "%s = %s %s %s\n", result, left, n->text, right);
```

More locations: `new_temp` at 95 to 100; assignment at 184 to 188; `if` at 190 to 202; `while` at 204 to 212; `for` at 214 to 231; I/O at 233 to 241. `emit` at 64 to 78 buffers instructions. `validate` at 109 to 126 rejects unsupported constructs.

Save TAC to a file for backup

```
.\minipascal.exe --tac .\presentation_demo\good.pas > tac.txt
Get-Content .\tac.txt
```

Demonstrate errors in execution and in code

Use the supplied files unchanged to reproduce these locations. Each example below returned exit code 1 in testing. Run \$LASTEXITCODE immediately after the compiler to show failure; the successful program returns 0.

Lexical error

```
program Demo;
begin
  @
end.

.\minipascal.exe --tac .\presentation_demo\lex.pas
lex error at 3:3: unexpected character '@'
```

Show compiler/lexer.l lines 166 to 170. The catch-all rule prints the unexpected character and increments error_count. This input has an invalid character.

Syntax error

```
program Demo
begin
end.

.\minipascal.exe --tac .\presentation_demo\syntax.pas
parse error at 2:1: syntax error
```

Show parser.y line 56 and lines 307 to 310. The program rule requires SEMI after the name. yyerror reports where parsing failed, here the begin token on line 2. The missing semicolon itself is on line 1. No TAC is printed.

Runtime error

```
program Demo;
var n: integer;
begin
  n := 10 div 0;
  writeln(n)
end.

.\minipascal.exe --run .\presentation_demo\zero.pas
runtime error at line 4: division by zero
```

Show interp.c lines 289 to 292. The arithmetic evaluator checks `r.num == 0` and calls `runtime_error`. The reporting function is at 46 to 49. Syntax is valid, so this error appears during execution.

Invalid runtime input

```
.\minipascal.exe --run .\presentation_demo\input.pas
```

Type abc and Enter. Verified message: **runtime error at line 4: expected a number on input.** Show `interp.c` lines 207 to 219, especially `scanf` and its return-value check.

Find rules and additional error handlers quickly

Navigate without memorizing line numbers

In VS Code: Ctrl+P, type compiler/parser.y, then Ctrl+G and enter 307. Ctrl+F searches the current file; Ctrl+Shift+F searches the project. In Notepad, open the file and use Ctrl+F for the search terms below.

```
Select-String -Path .\compiler\*.c,.\compiler\*.l,`
.\compiler\*.y -Pattern 'yyerror|runtime_error|emit_tac'
```

Lexer and parser rules to point at

lexer.l: 52 to 53 define DIGIT and ID; 87 to 114 map keywords; 116 maps := to ASSIGN; 138 to 155 handle literals and identifiers. Keywords precede the identifier rule; matching is case-insensitive at line 48. Locations are tracked at 17 to 21.

parser.y: 55 to 64 program; 86 to 92 declaration; 158 to 168 begin/end and statement sequence; 175 to 187 if/else; 221 to 229 assignment; 272 to 302 expressions; 42 to 49 precedence. Search the rule name.

```
program : PROGRAM ident SEMI decls subprograms compound DOT
compound : BEGIN_KW stmts END
assign_or_call : ident ASSIGN expr
expr : expr PLUS expr /* one alternative */
```

The C action after a grammar alternative creates an AST node. In the assignment action, \$\$ is the result node, \$1 is the identifier, and \$3 is the expression. @2.first_line carries the operator location. ast.c lines 16 to 20 allocate nodes; lines 56 to 62 build binary-operation nodes.

Other errors and verified commands

```
.\minipascal.exe --run .\presentation_demo\unknown.pas
```

unknown.pas assigns x without a declaration. Verified: **runtime error at line 3: unknown variable**. Show interp.c 118 to 129, search unknown variable. This is a runtime lookup check, not a separate static semantic-analysis pass.

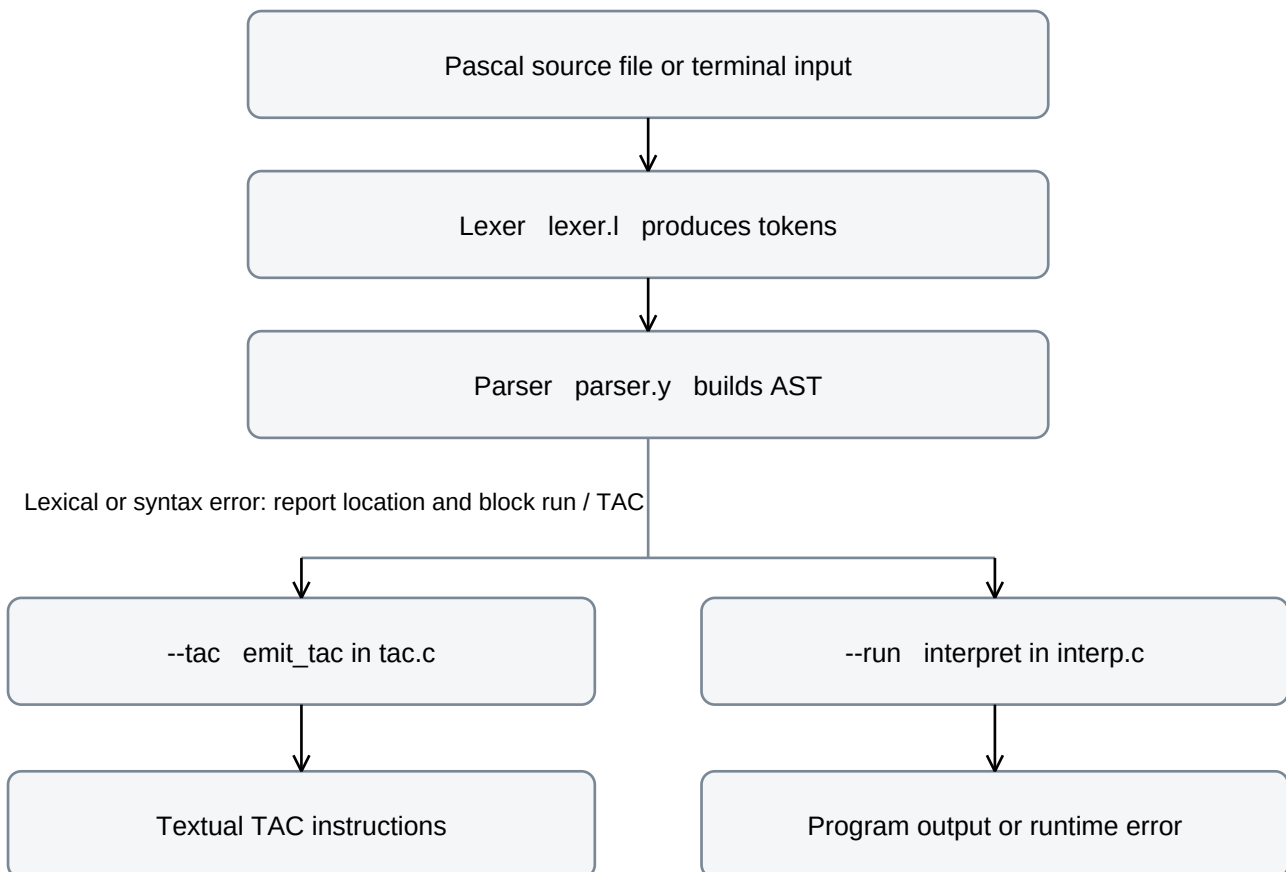
```
.\minipascal.exe --run .\presentation_demo\array.pas
.\minipascal.exe --tac .\presentation_demo\array.pas
```

The file declares a[1..2] and assigns a[3]. --run reports **array index out of range** at line 4 (interp.c 131 to 146). --tac reports **arrays are not supported by TAC generation yet** at line 2 (tac.c 113 to 114). Runtime array support and TAC support differ.

Additional handlers in source: unterminated comments in lexer.l 68 to 84; unterminated strings at 158 to 163; array needs an index / not an array in interp.c 131 to 146; numeric operands at 270 to 276; unknown function at 300 to 305; unknown procedure at 342 to 345. TAC fail is at tac.c 31 to 34.

Parser recovery uses error and yyerrok at parser.y 218. Do not claim it recovers cleanly from every malformed program. Use the verified syntax example during the short demonstration.

Compiler flowchart



What to say while pointing

"Source characters go to the Flex lexer, which produces tokens. The Bison parser matches grammar rules and creates an AST. If the frontend reports an error, execution and TAC generation are blocked. For valid input, --tac traverses the AST to print intermediate instructions, while --run interprets the AST to produce the program result."

Build flow versus program flow

Build time: parser.y goes through Bison to parser.tab.c and parser.tab.h. lexer.l goes through Flex to lex.yy.c. GCC links these with ast.c, dump.c, interp.c, tac.c, and main.c to create minipascal.exe.

Program processing time: minipascal.exe reads your .pas input. Its selected mode determines whether you see tokens, the AST, textual TAC, or runtime output. A separate typed machine-code backend is not implemented here.

Error flow

Lexer and parser errors increment error_count. main.c lines 145 to 150 use it to prevent --run and --tac. TAC validation rejects unsupported AST constructs before printing instructions. Runtime checks increase run_errors, and interpret returns a failure status. Some execution side effects can occur before a runtime error is detected.

Short speaking script and Q and A

Rehearsal script

"This is our MiniPascal project, built using Flex, Bison, and C. I first show `lexer.l`, then build the project to create `minipascal.exe`. This executable processes Pascal source as input."

"My program sets `n` to 5. If `n` is positive, it subtracts one and prints `n`. I run it with `--run`, and the output is 4. With `--tac`, the same source produces a comparison temporary, a conditional jump, a subtraction temporary, and write instructions."

"Now I remove the program-header semicolon. The parser reports a syntax error at the next token. The required semicolon is visible in the program rule, and `yyerror` prints the location. For a valid program containing division by zero, the interpreter reports a runtime error. Here is the zero-divisor check in `interp.c`."

"The grammar rules are in `parser.y`. Their actions create AST nodes. TAC is generated in `tac.c`: expression creates temporaries, statements handles assignments and branches, and `emit_tac` prints the result. Our flow is source to tokens to AST, then either textual TAC or AST interpretation."

Likely questions and short answers

Where is `ir.c`? This project names its IR module `tac.c`. `main.c` calls `emit_tac`; expression and statements perform the lowering.

Why three-address? A typical instruction has a destination and two source operands, such as `t2 = n - 1`. Temporary values break complex expressions into simple steps.

Does `--run` execute the TAC? No. It calls `interpret` on the AST. TAC output is a separate demonstration of IR generation.

Is type checking complete? No separate static semantic pass is implemented. There are runtime checks, but TAC does not validate declarations or type compatibility. Numeric runtime values use C `double`.

What is precedence? `parser.y` declares it at 42 to 49. Multiplication binds more tightly than addition, so `2 + 3 * 4` groups as `2 + (3 * 4)`.

How are errors located? The lexer records line and column in `yylloc`. `yyerror` reports those coordinates. AST nodes store line numbers for runtime and TAC errors.

Can every parsed feature generate TAC? No. Arrays and user subprograms are examples rejected by TAC validation, even though the parser recognizes them.

Final rehearsal checklist

Open `good.pas`, `lexer.l`, `parser.y`, `interp.c`, and `tac.c` in advance. Rehearse the exact commands on your presentation terminal. Keep this PDF and the tested sample files available. For three minutes, show only the success program, TAC, syntax error, and division error; use the other examples for Q and A.

Verification: the current generated C sources and handwritten modules compiled successfully with GCC. The success, TAC, token, tree, and error examples were executed against that rebuilt test executable. Full Flex/Bison regeneration and interactive Ctrl+Z entry were not verified in this environment.